

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: df = pd.read_csv('housing.csv')
```

```
In [ ]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   longitude              20640 non-null  float64
1   latitude               20640 non-null  float64
2   housing_median_age     20640 non-null  float64
3   total_rooms            20640 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20640 non-null  float64
6   households              20640 non-null  float64
7   median_income          20640 non-null  float64
8   median_house_value     20640 non-null  float64
9   ocean_proximity        20640 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.6 MB
```

```
In [ ]: df.head(5)
```

Out[]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_hou
0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	



In []: `df.isnull().sum()`

Out[]:

```

longitude      0
latitude       0
housing_median_age  0
total_rooms    0
total_bedrooms  0
population     0
households     0
median_income  0
median_house_value  0
ocean_proximity  0
dtype: int64

```

In []: `df.dropna(axis=0 , inplace=True)`

In []: `df.info()`

```

<class 'pandas.DataFrame'>
Index: 20433 entries, 0 to 20639
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   longitude              20433 non-null  float64
1   latitude               20433 non-null  float64
2   housing_median_age     20433 non-null  float64
3   total_rooms            20433 non-null  float64
4   total_bedrooms         20433 non-null  float64
5   population             20433 non-null  float64
6   households              20433 non-null  float64
7   median_income          20433 non-null  float64
8   median_house_value     20433 non-null  float64
9   ocean_proximity        20433 non-null  str
dtypes: float64(9), str(1)
memory usage: 1.7 MB

```

```
In [ ]: df = pd.get_dummies(df , columns=['ocean_proximity'] , dtype=int)
```

```
In [ ]: df.head(5)
```

```
Out [ ]:
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_hou
--	-----------	----------	--------------------	-------------	----------------	------------	------------	---------------	------------

0	-122.23	37.88	41.0	880.0	129.0	322.0	126.0	8.3252	
1	-122.22	37.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	
2	-122.24	37.85	52.0	1467.0	190.0	496.0	177.0	7.2574	
3	-122.25	37.85	52.0	1274.0	235.0	558.0	219.0	5.6431	
4	-122.25	37.85	52.0	1627.0	280.0	565.0	259.0	3.8462	



```
In [ ]: df.corr()
```

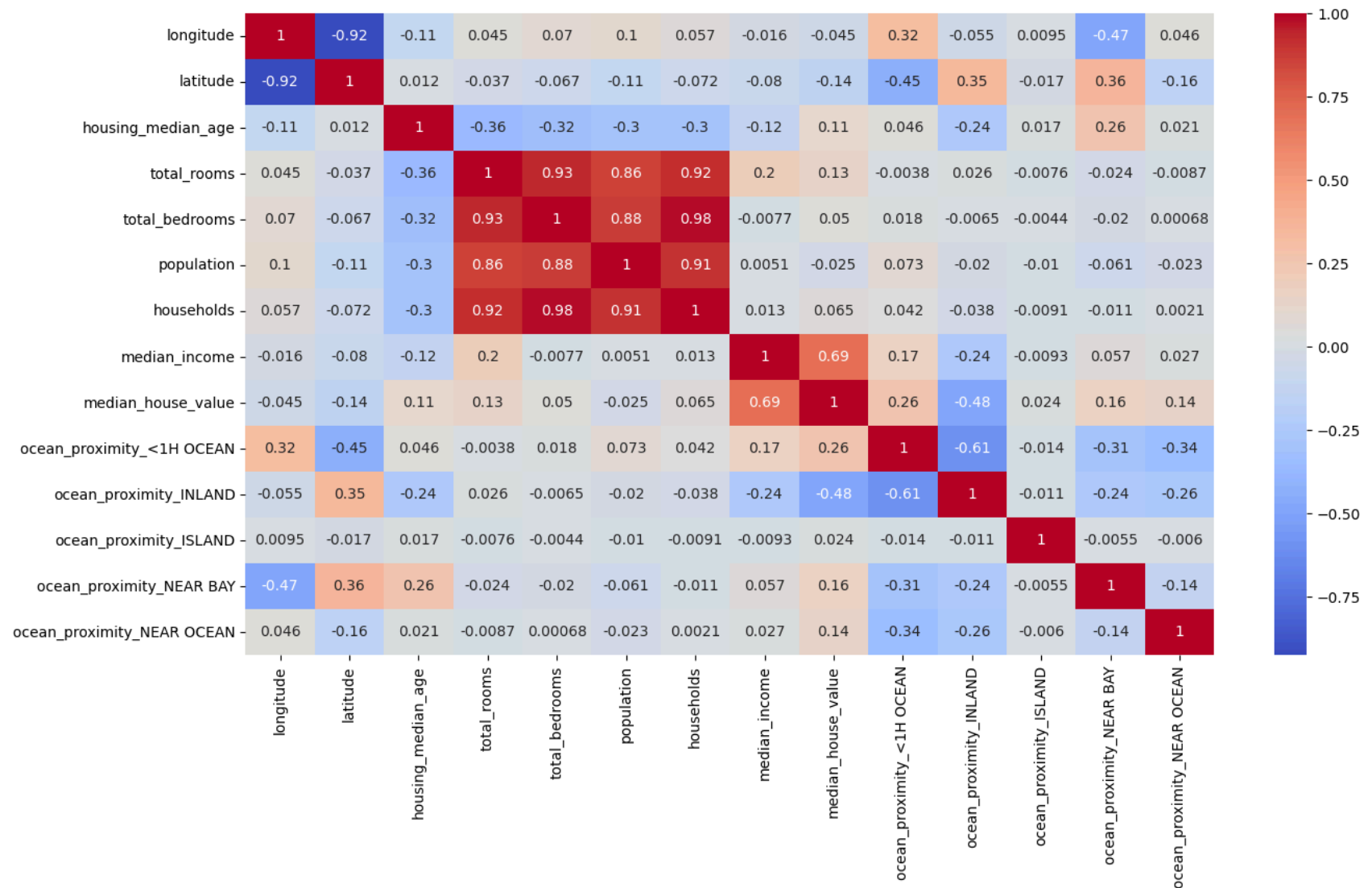
Out[]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income
longitude	1.000000	-0.924616	-0.109357	0.045480	0.069608	0.100270	0.056513	
latitude	-0.924616	1.000000	0.011899	-0.036667	-0.066983	-0.108997	-0.071774	
housing_median_age	-0.109357	0.011899	1.000000	-0.360628	-0.320451	-0.295787	-0.302768	
total_rooms	0.045480	-0.036667	-0.360628	1.000000	0.930380	0.857281	0.918992	
total_bedrooms	0.069608	-0.066983	-0.320451	0.930380	1.000000	0.877747	0.979728	
population	0.100270	-0.108997	-0.295787	0.857281	0.877747	1.000000	0.907186	
households	0.056513	-0.071774	-0.302768	0.918992	0.979728	0.907186	1.000000	
median_income	-0.015550	-0.079626	-0.118278	0.197882	-0.007723	0.005087	0.013434	
median_house_value	-0.045398	-0.144638	0.106432	0.133294	0.049686	-0.025300	0.064894	
ocean_proximity_<1H OCEAN	0.320831	-0.446928	0.045553	-0.003777	0.018314	0.073450	0.041883	
ocean_proximity_INLAND	-0.055337	0.351084	-0.236968	0.026477	-0.006463	-0.019602	-0.038265	
ocean_proximity_ISLAND	0.009501	-0.016662	0.017105	-0.007603	-0.004361	-0.010451	-0.009119	
ocean_proximity_NEAR BAY	-0.474714	0.358785	0.256149	-0.023647	-0.019873	-0.061480	-0.011280	
ocean_proximity_NEAR OCEAN	0.046185	-0.161342	0.020797	-0.008674	0.000679	-0.023498	0.002106	



```
In [ ]: plt.figure(figsize=(15,8))
sns.heatmap(df.corr() , annot=True , cmap="coolwarm")
```

Out[]: <Axes: >



$|\text{corr}(x1, x2)| > 0.7$

ratio , difference, product

```
In [ ]: df['bed/room'] = df['total_rooms']/df['total_bedrooms']
df['troom/house'] = df['households']/df['total_rooms']
df['location_lat*lon'] = df['latitude']*df['longitude']
```

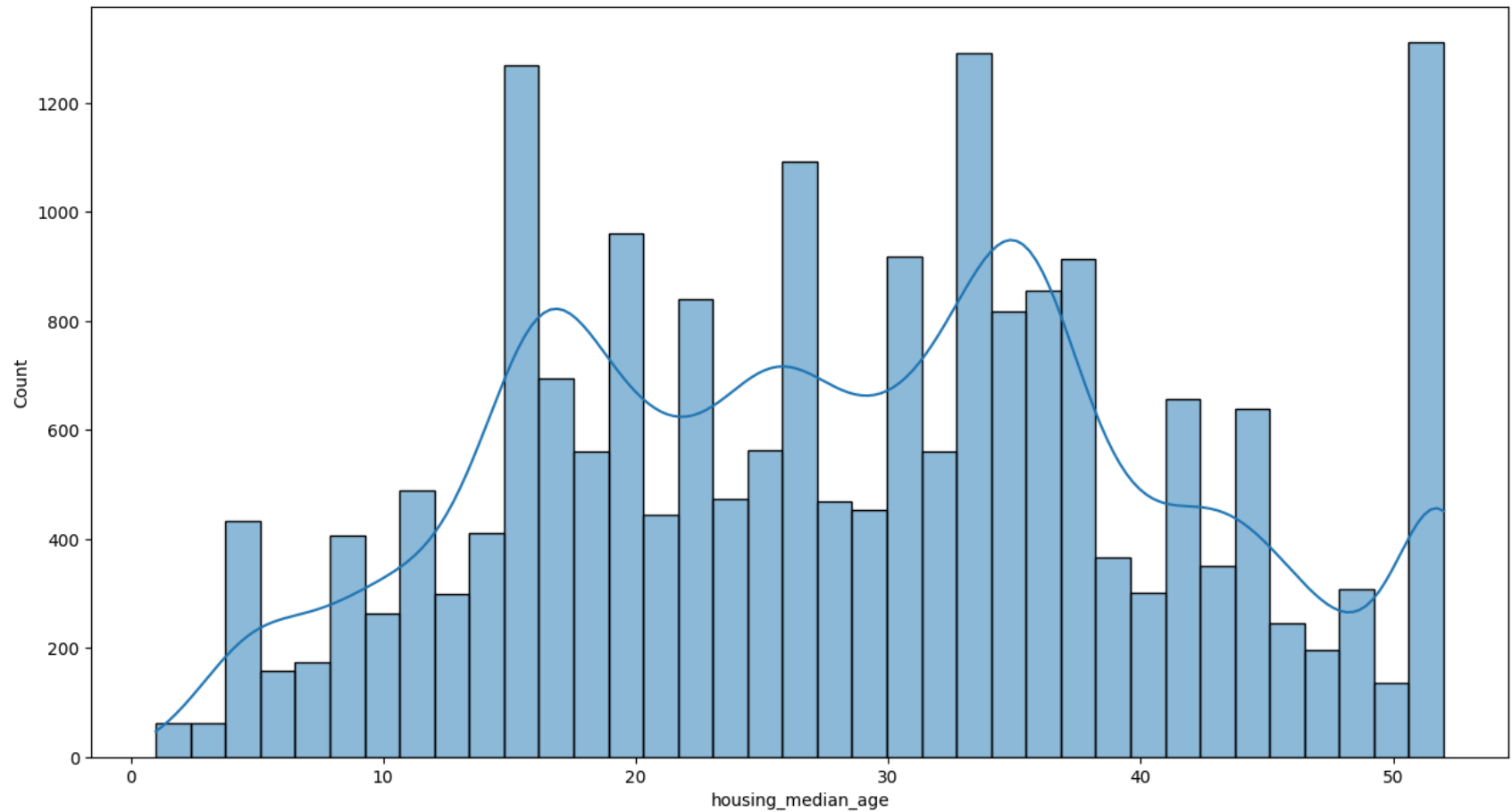
```
In [ ]: df.info()
```

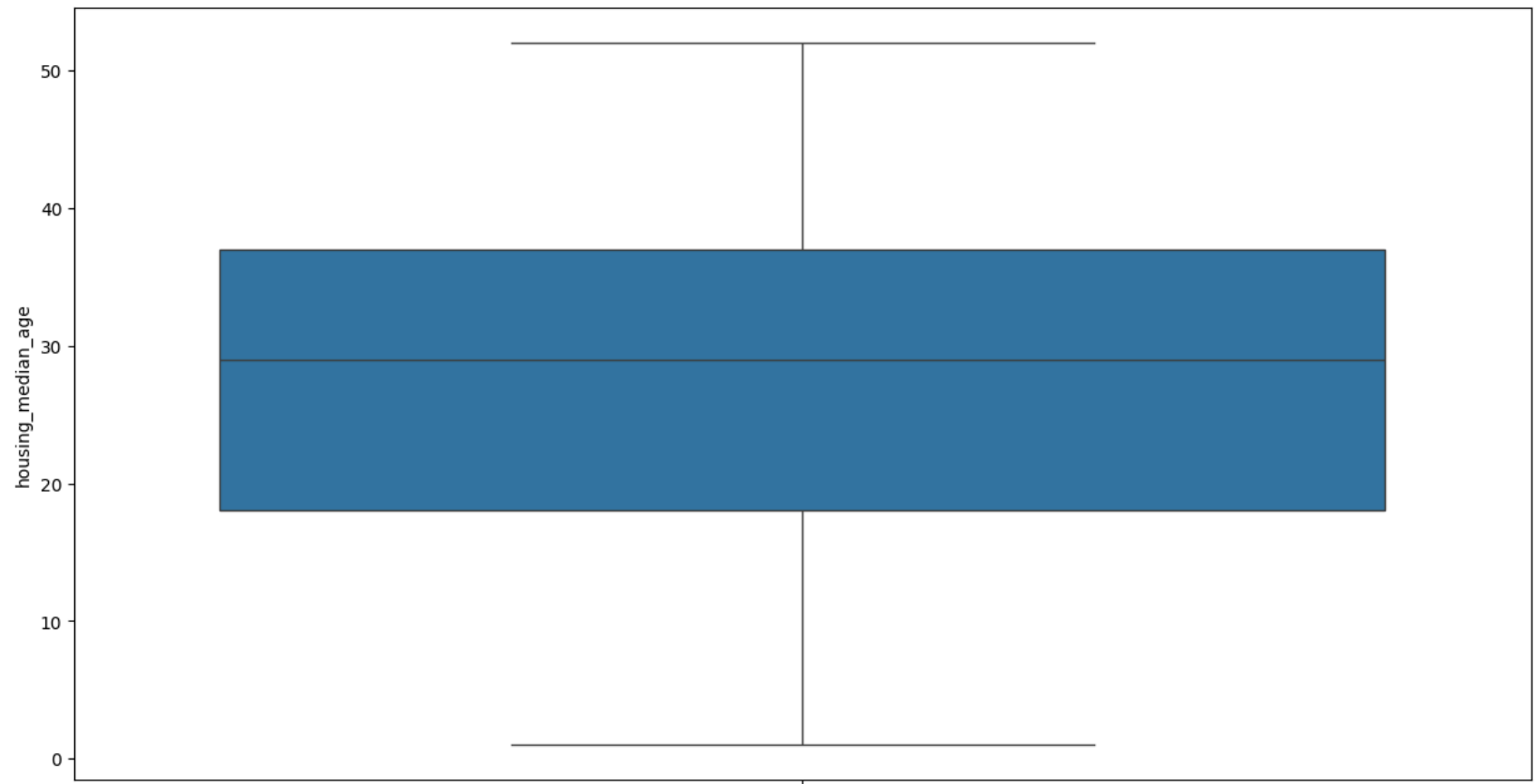
```
<class 'pandas.DataFrame'>
Index: 20433 entries, 0 to 20639
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   longitude                             20433 non-null  float64
1   latitude                              20433 non-null  float64
2   housing_median_age                    20433 non-null  float64
3   total_rooms                           20433 non-null  float64
4   total_bedrooms                        20433 non-null  float64
5   population                            20433 non-null  float64
6   households                             20433 non-null  float64
7   median_income                         20433 non-null  float64
8   median_house_value                    20433 non-null  float64
9   ocean_proximity_<1H OCEAN            20433 non-null  int64
10  ocean_proximity_INLAND                20433 non-null  int64
11  ocean_proximity_ISLAND                20433 non-null  int64
12  ocean_proximity_NEAR BAY              20433 non-null  int64
13  ocean_proximity_NEAR OCEAN            20433 non-null  int64
14  bed/room                              20433 non-null  float64
15  troom/house                           20433 non-null  float64
16  location_lat*lon                       20433 non-null  float64
dtypes: float64(12), int64(5)
memory usage: 2.8 MB
```

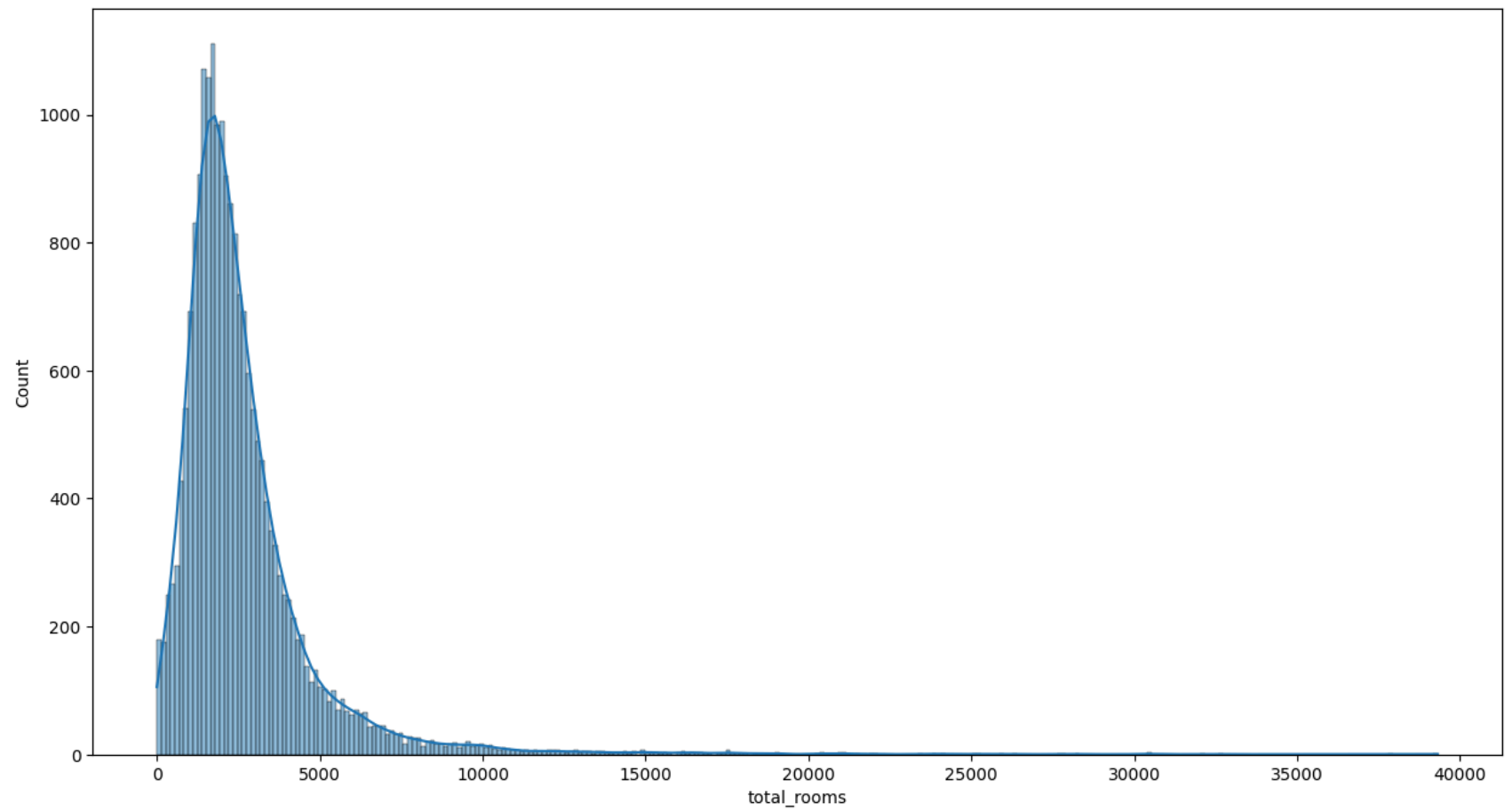
```
In [ ]: df.columns
```

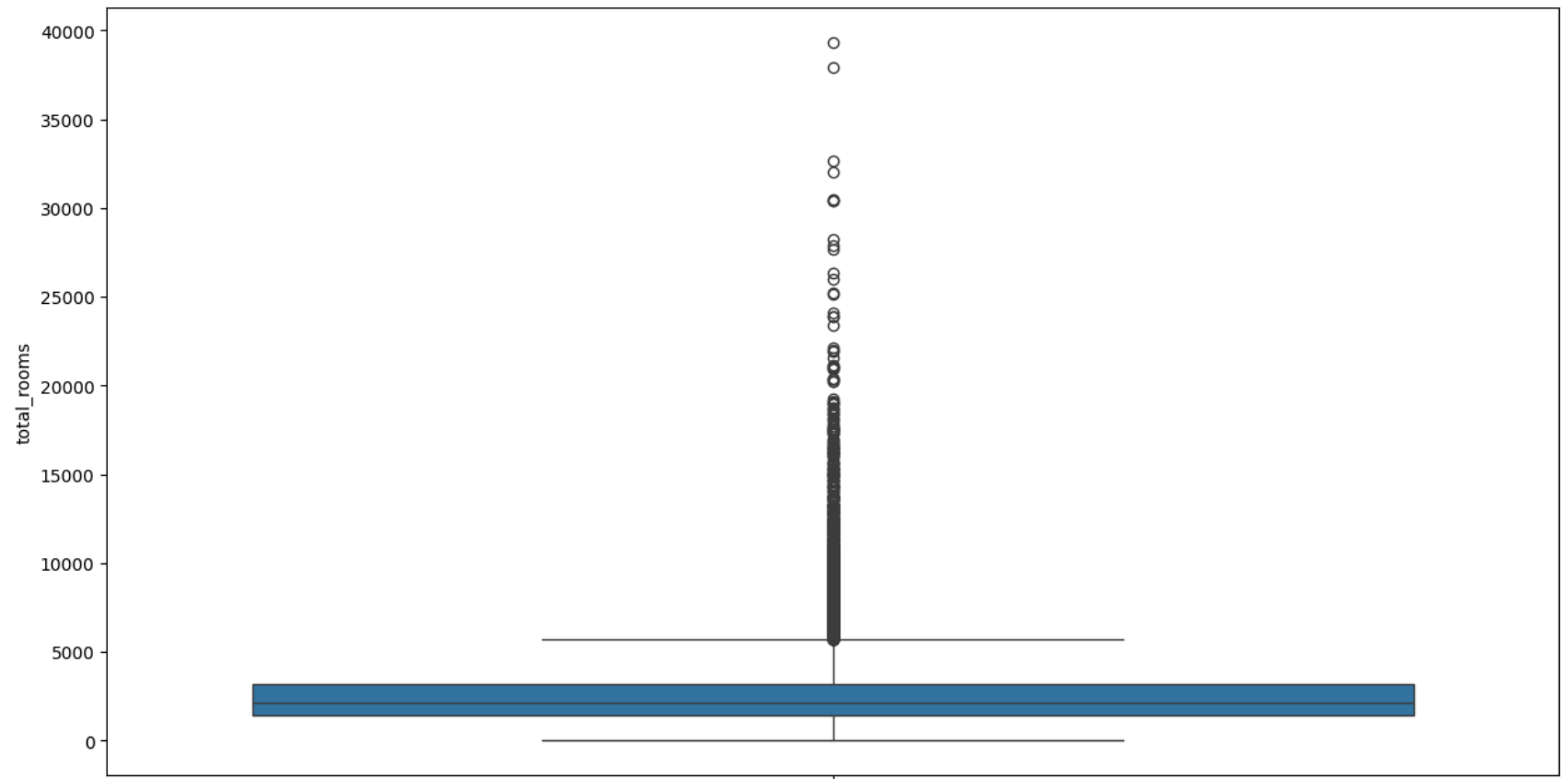
```
Out[ ]: Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
              'total_bedrooms', 'population', 'households', 'median_income',
              'median_house_value', 'ocean_proximity_<1H OCEAN',
              'ocean_proximity_INLAND', 'ocean_proximity_ISLAND',
              'ocean_proximity_NEAR BAY', 'ocean_proximity_NEAR OCEAN', 'bed/room',
              'pop/house', 'troom/house', 'location_lat*lon'],
              dtype='object')
```

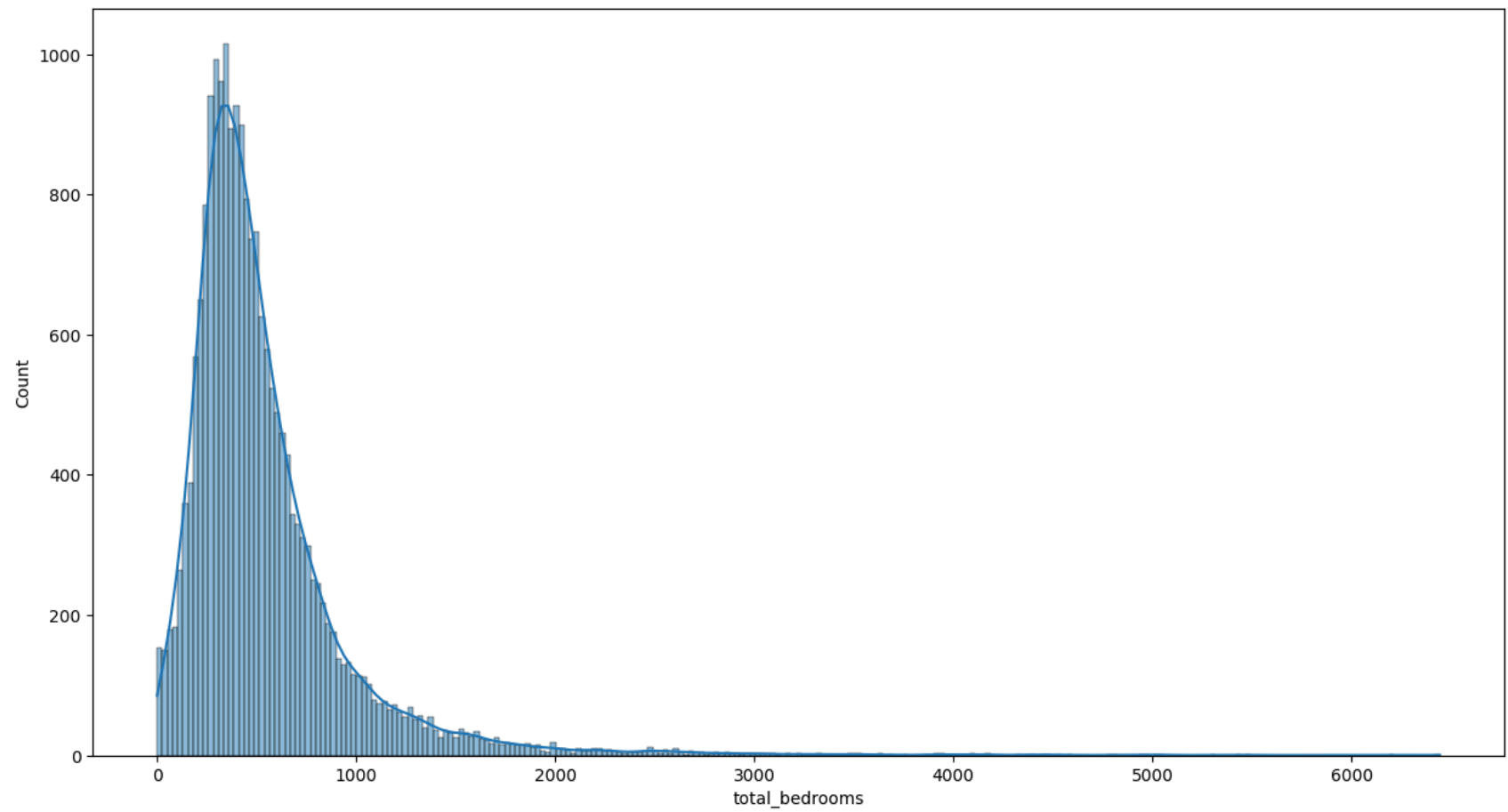
```
In [ ]: for col in ['housing_median_age', 'total_rooms',  
                  'total_bedrooms', 'population', 'households', 'median_income',  
                  'median_house_value', 'bedroom',  
                  'troom/house', 'location_lat*lon']:  
  
    plt.figure(figsize=(15,8))  
    sns.histplot(df[col] , kde=True)  
    plt.show()  
  
    plt.figure(figsize=(15,8))  
    sns.boxplot(df[col])  
    plt.show()
```

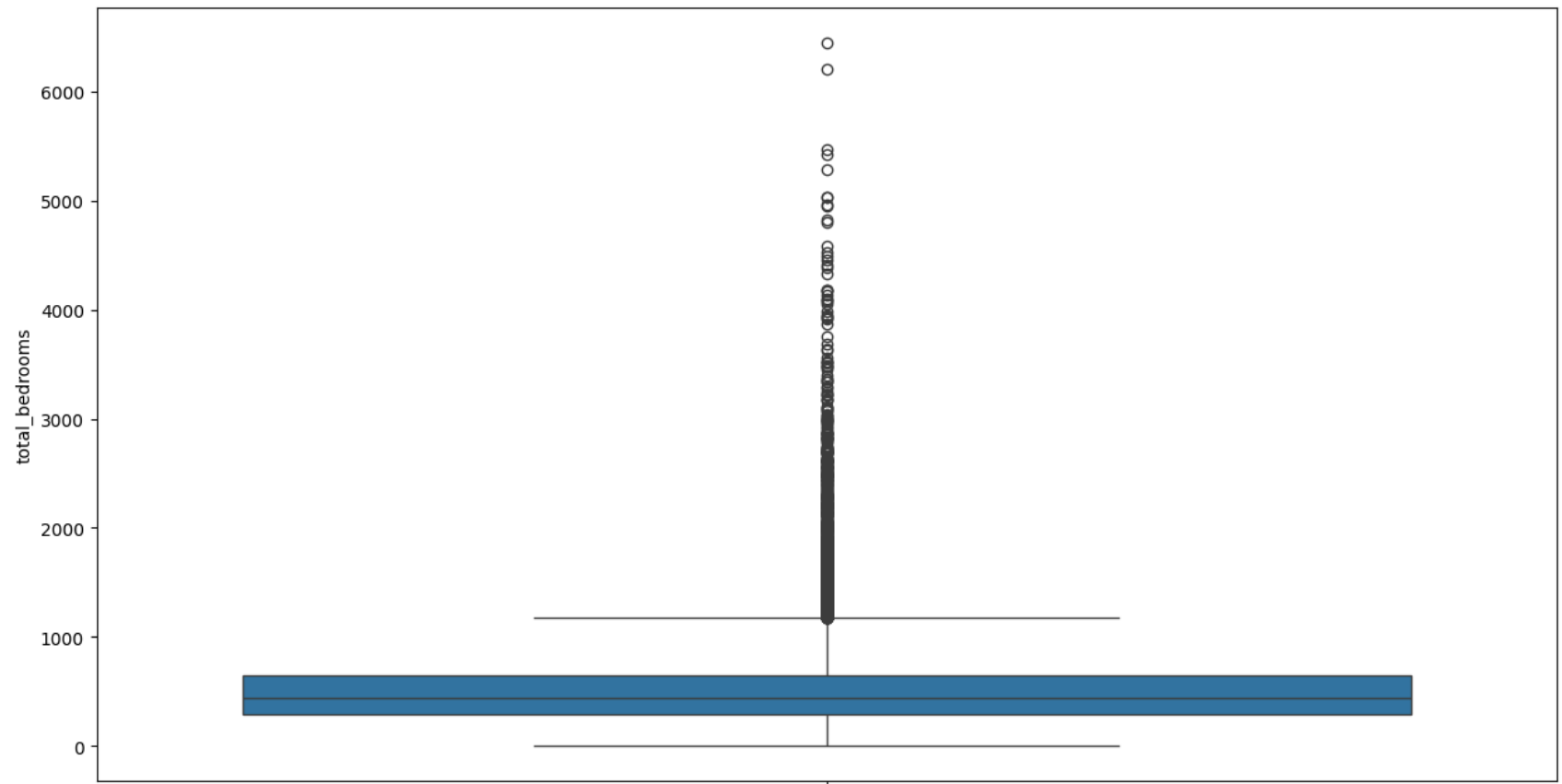


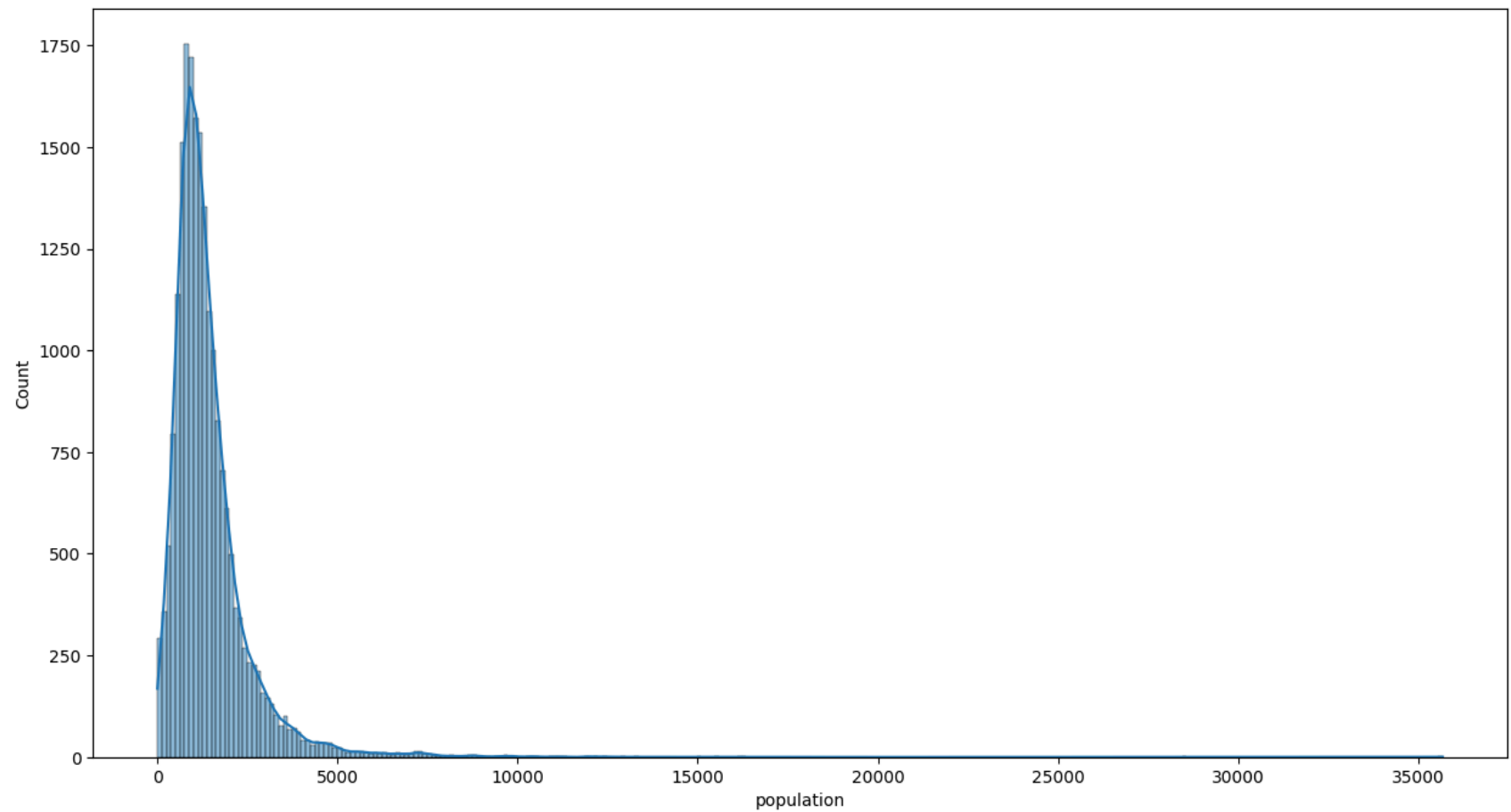


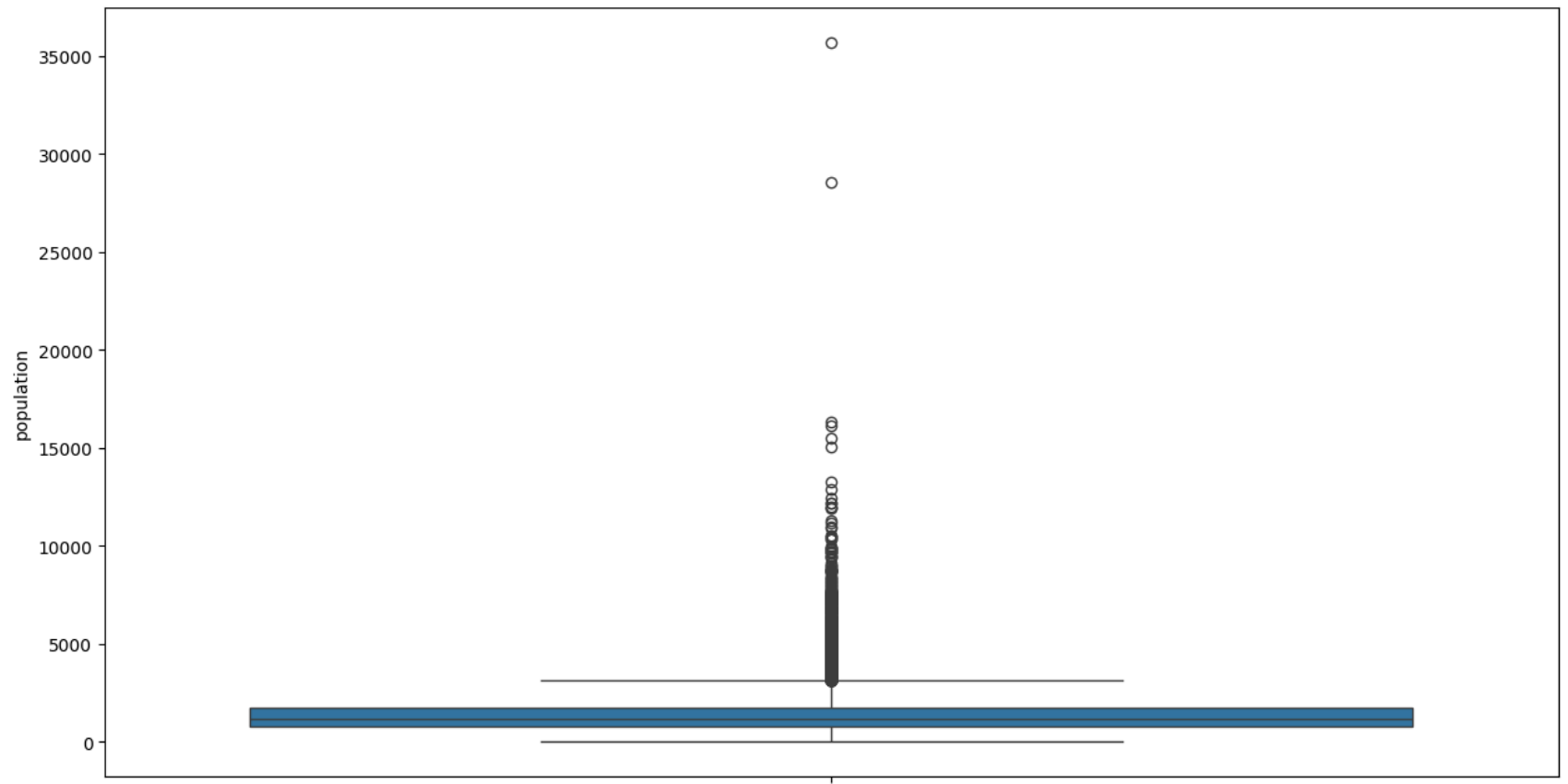


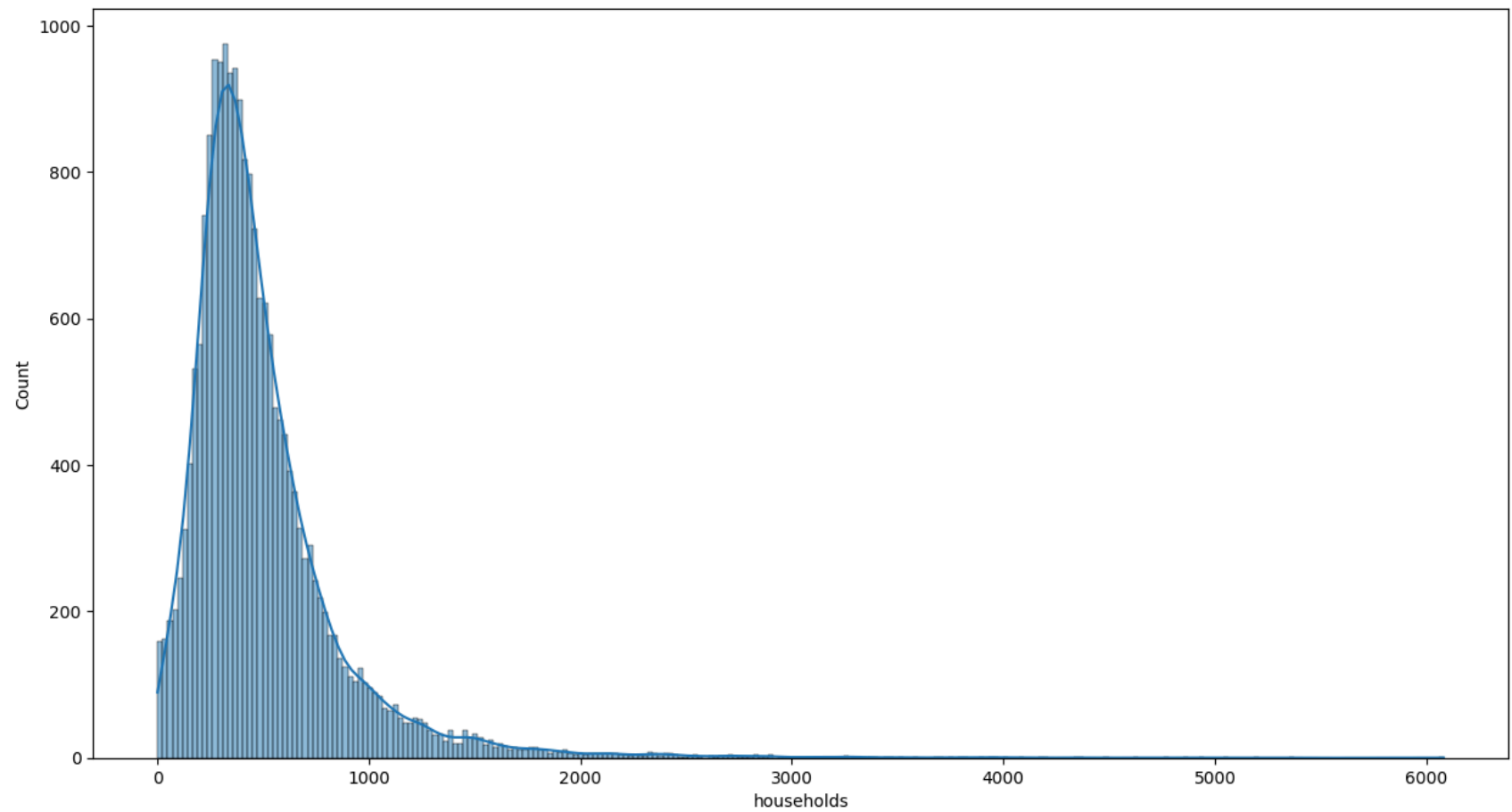


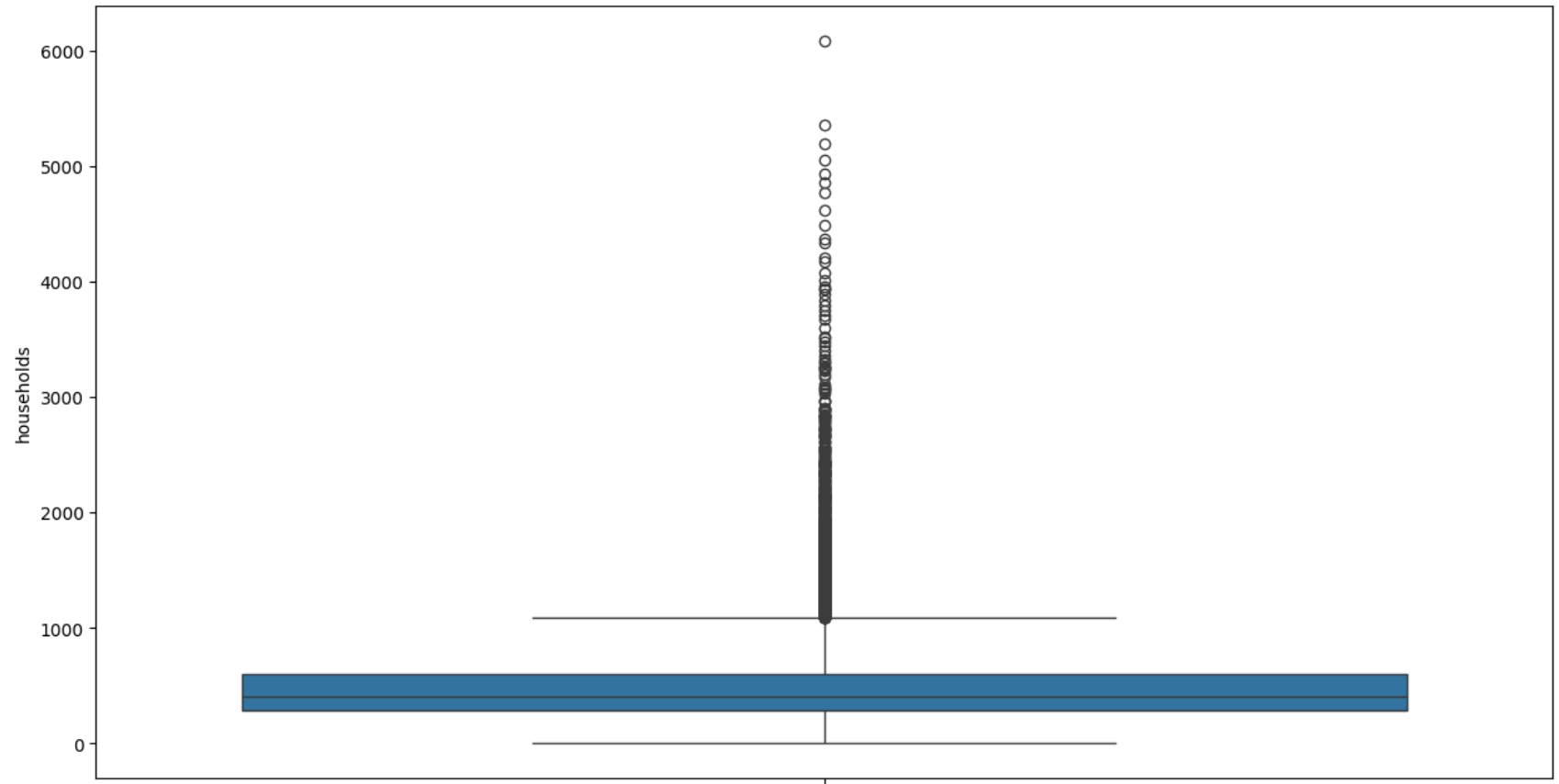


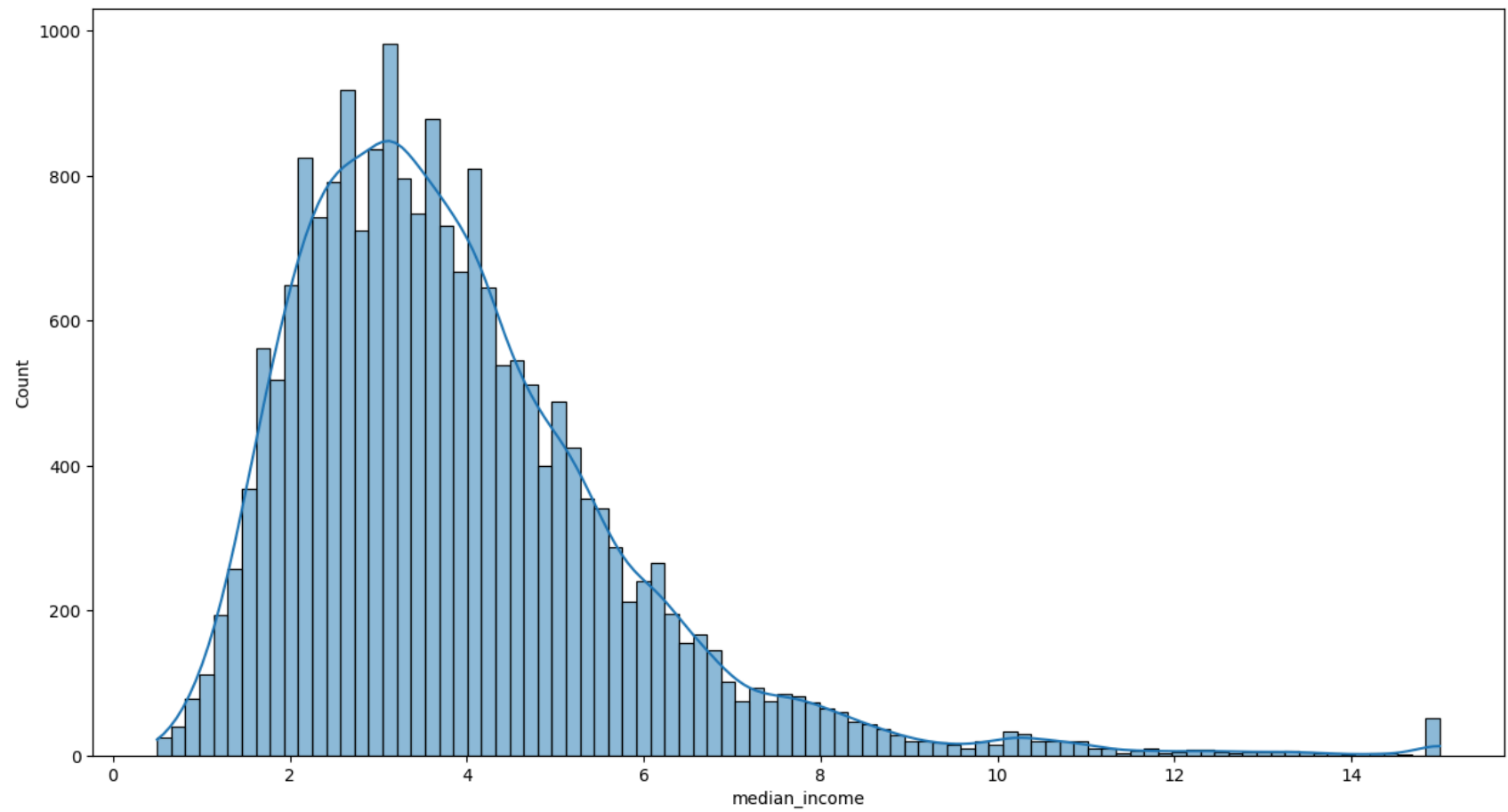


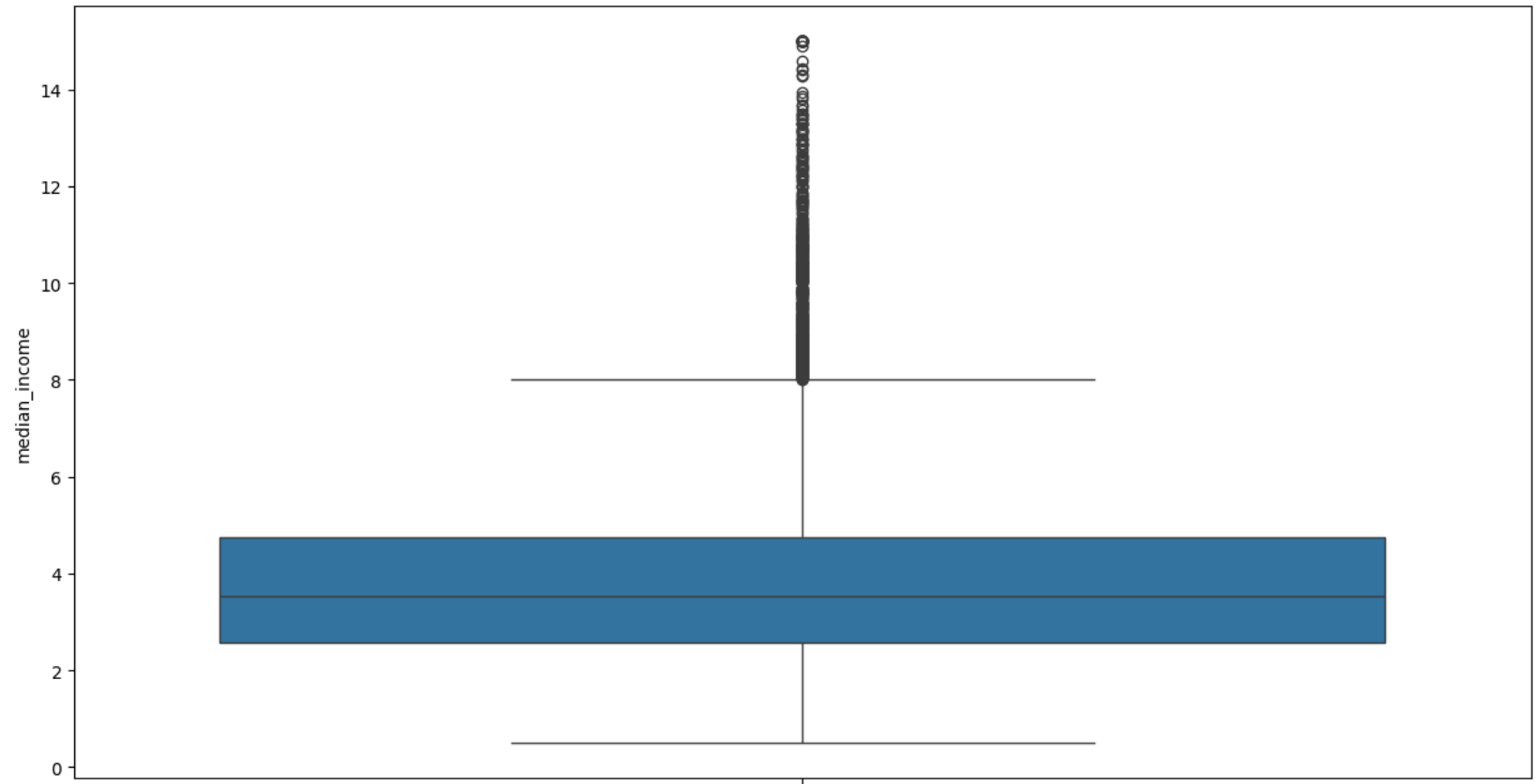


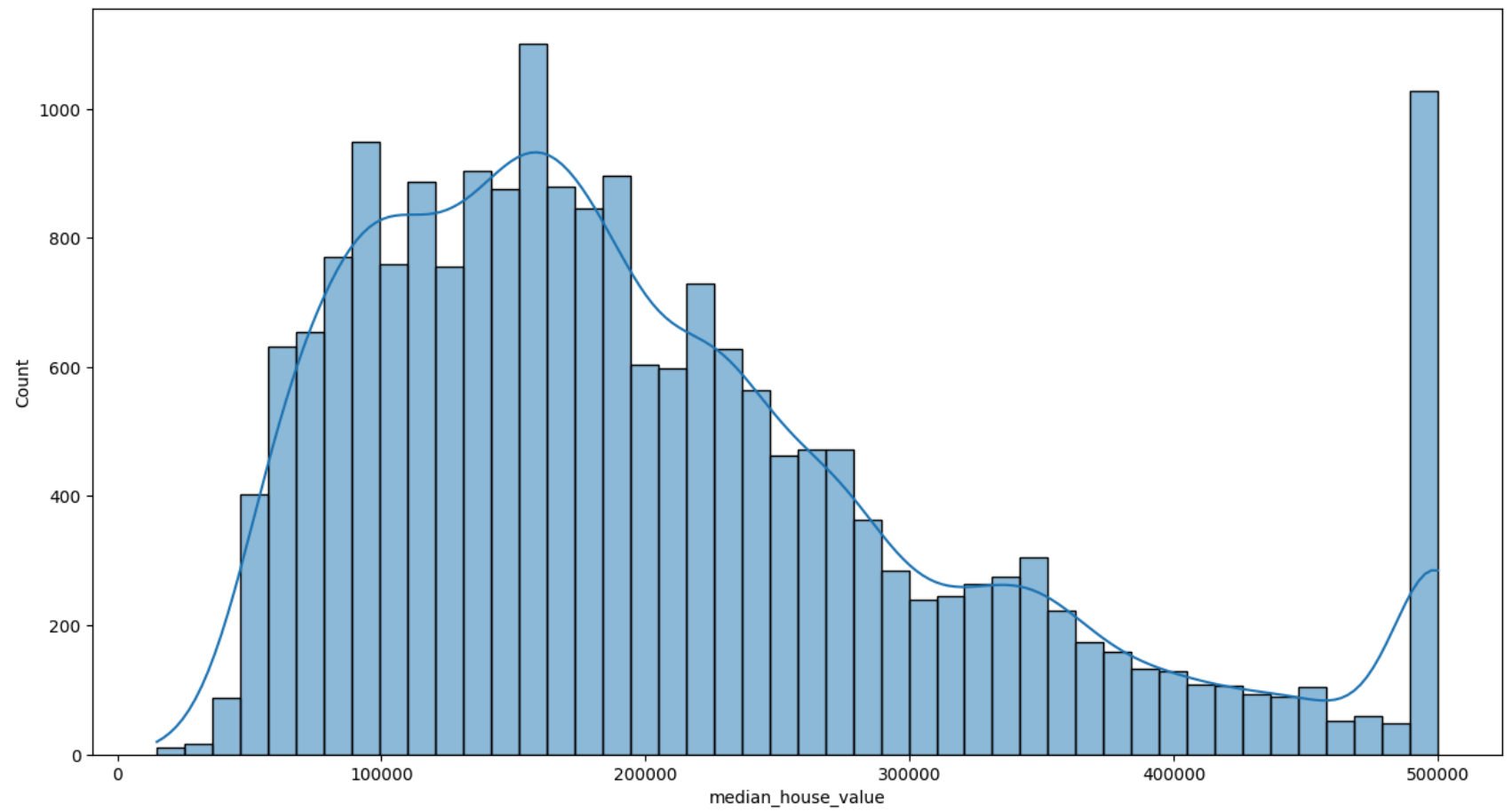


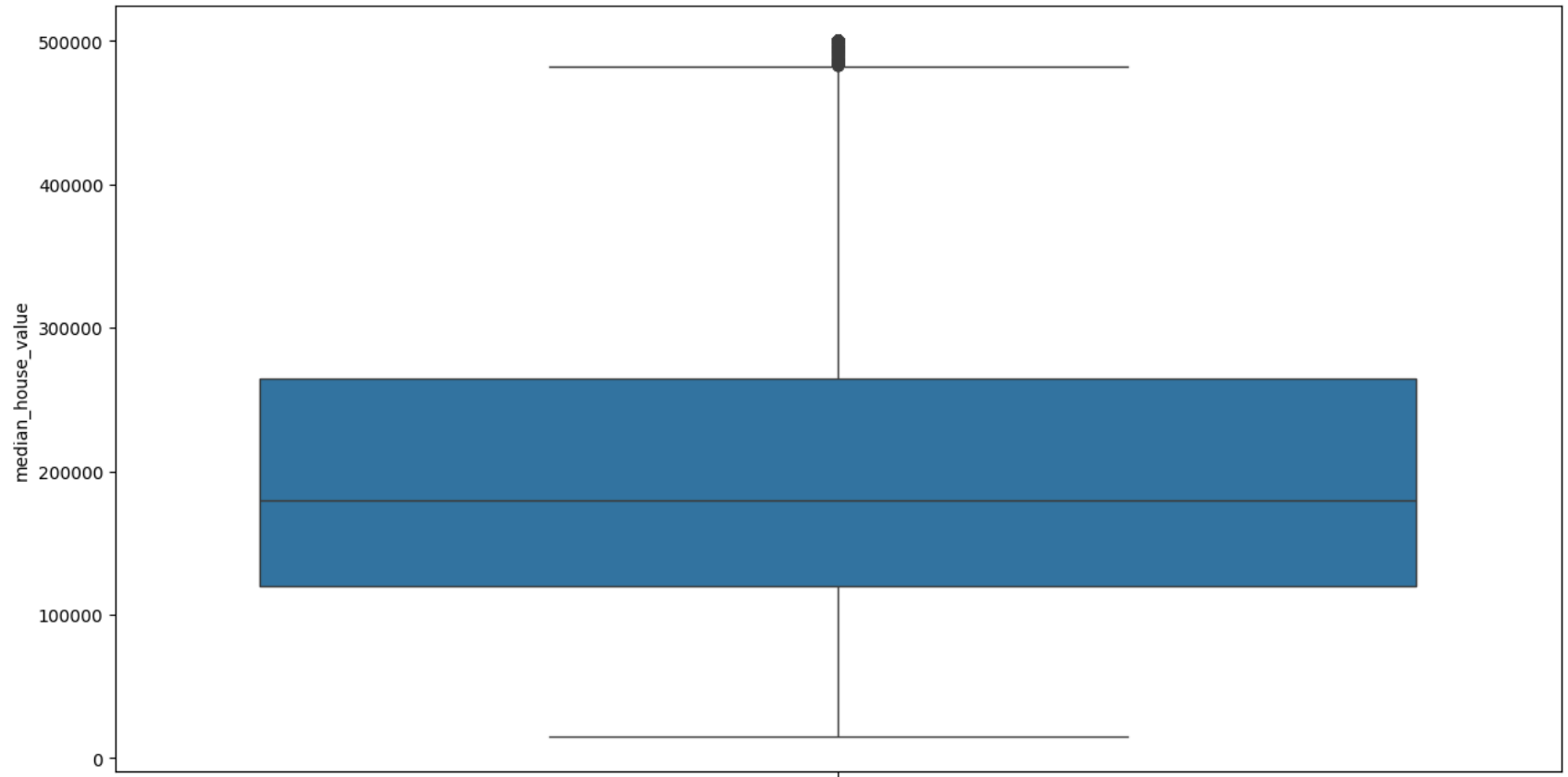


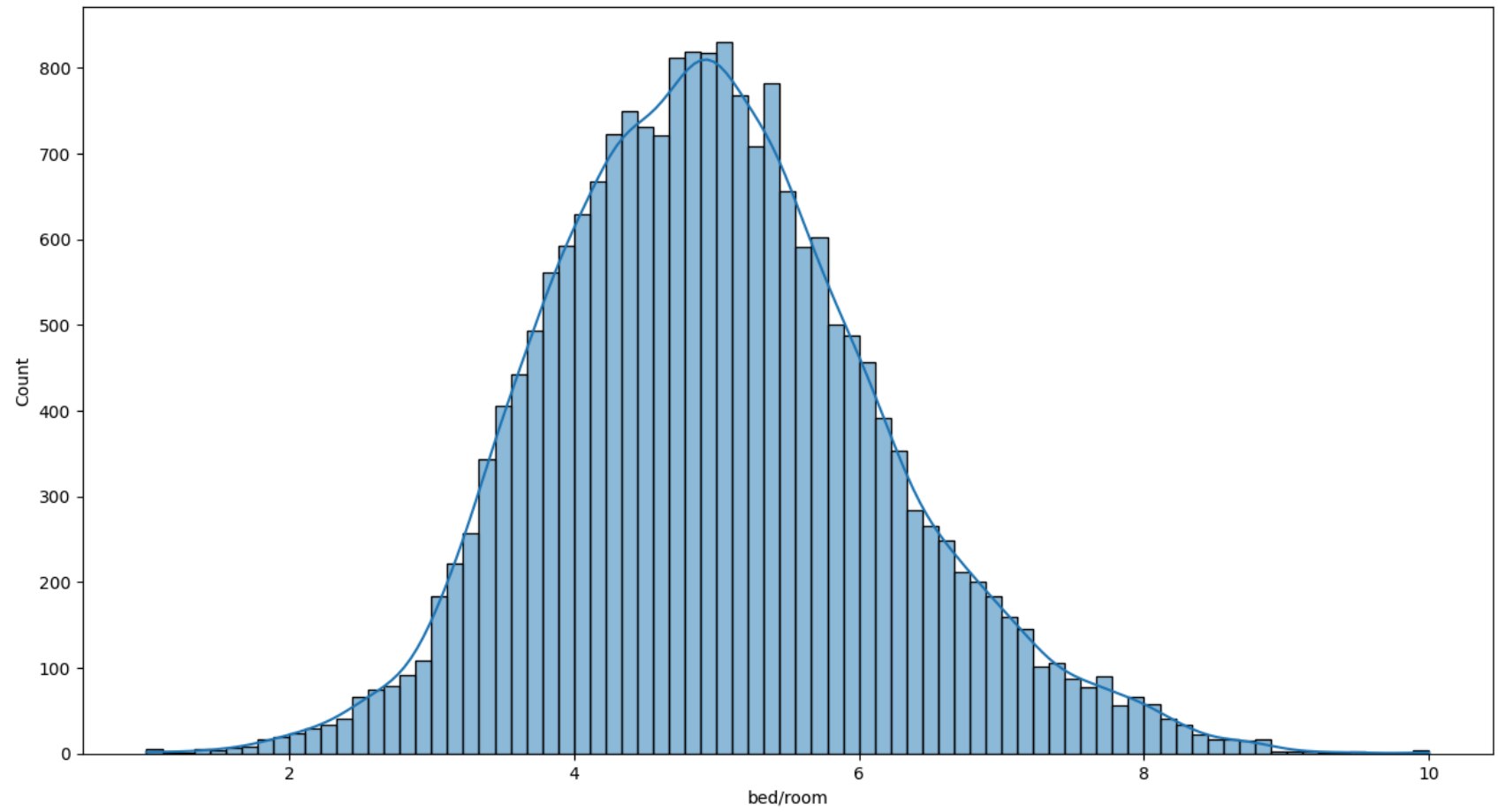


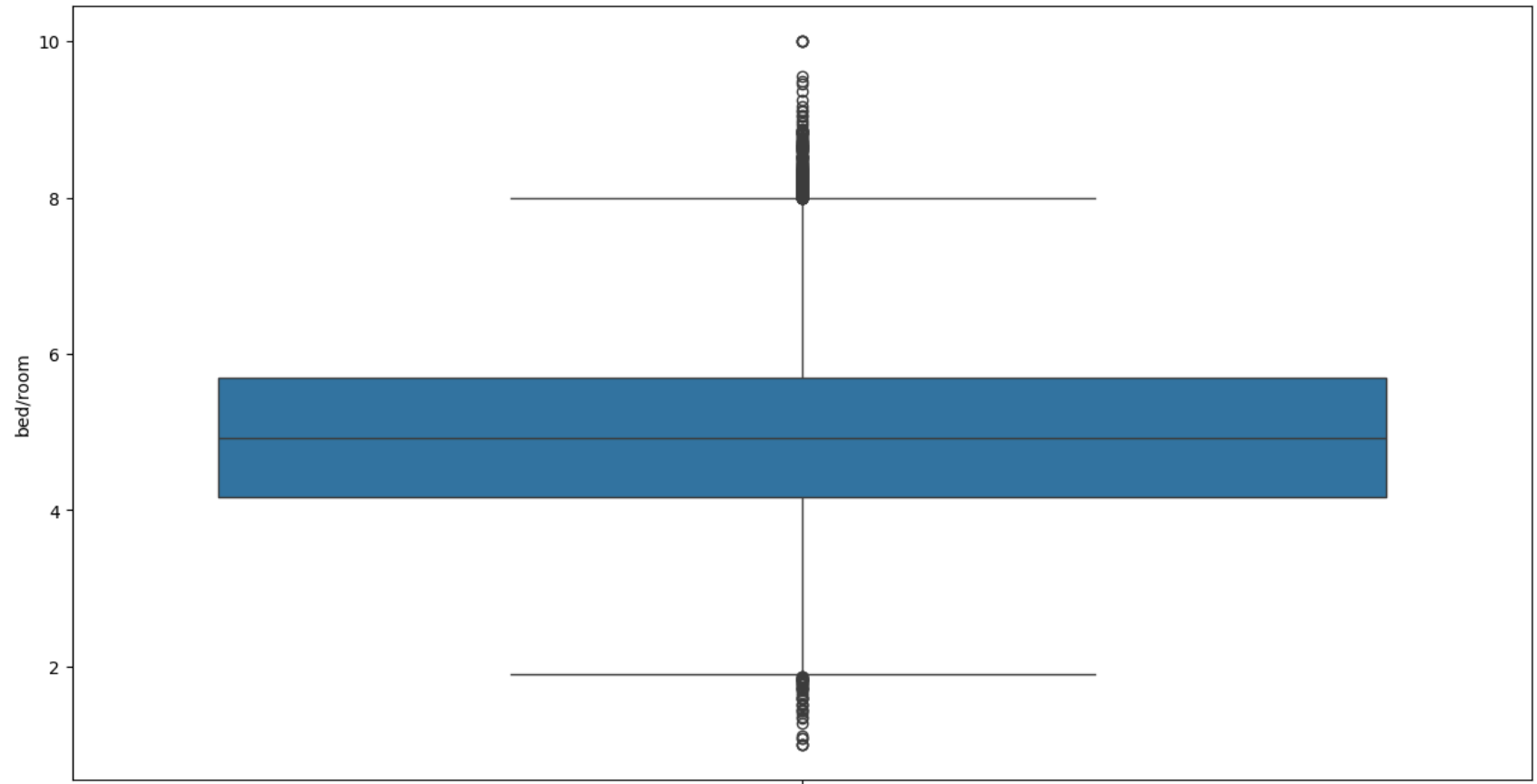


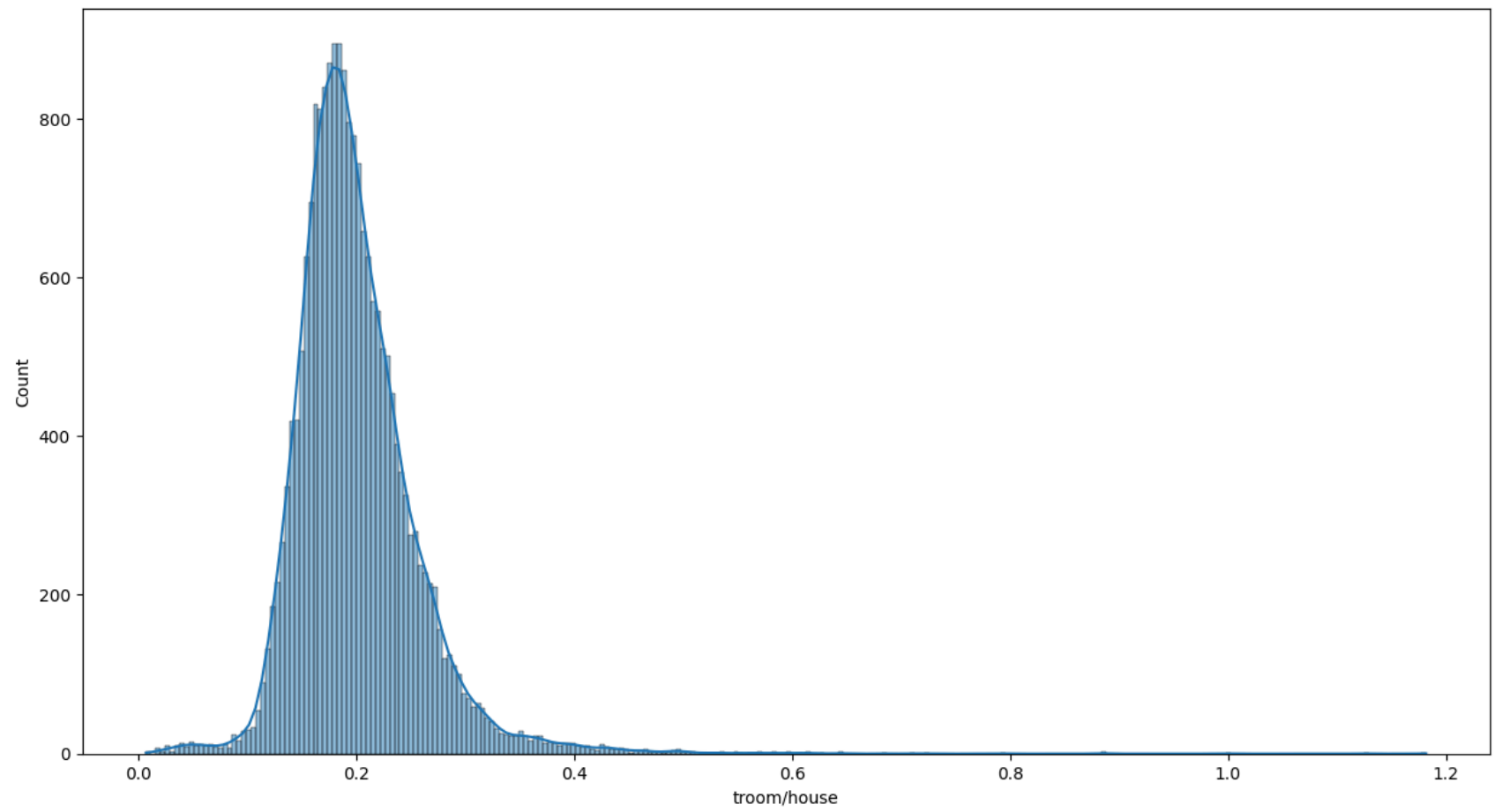


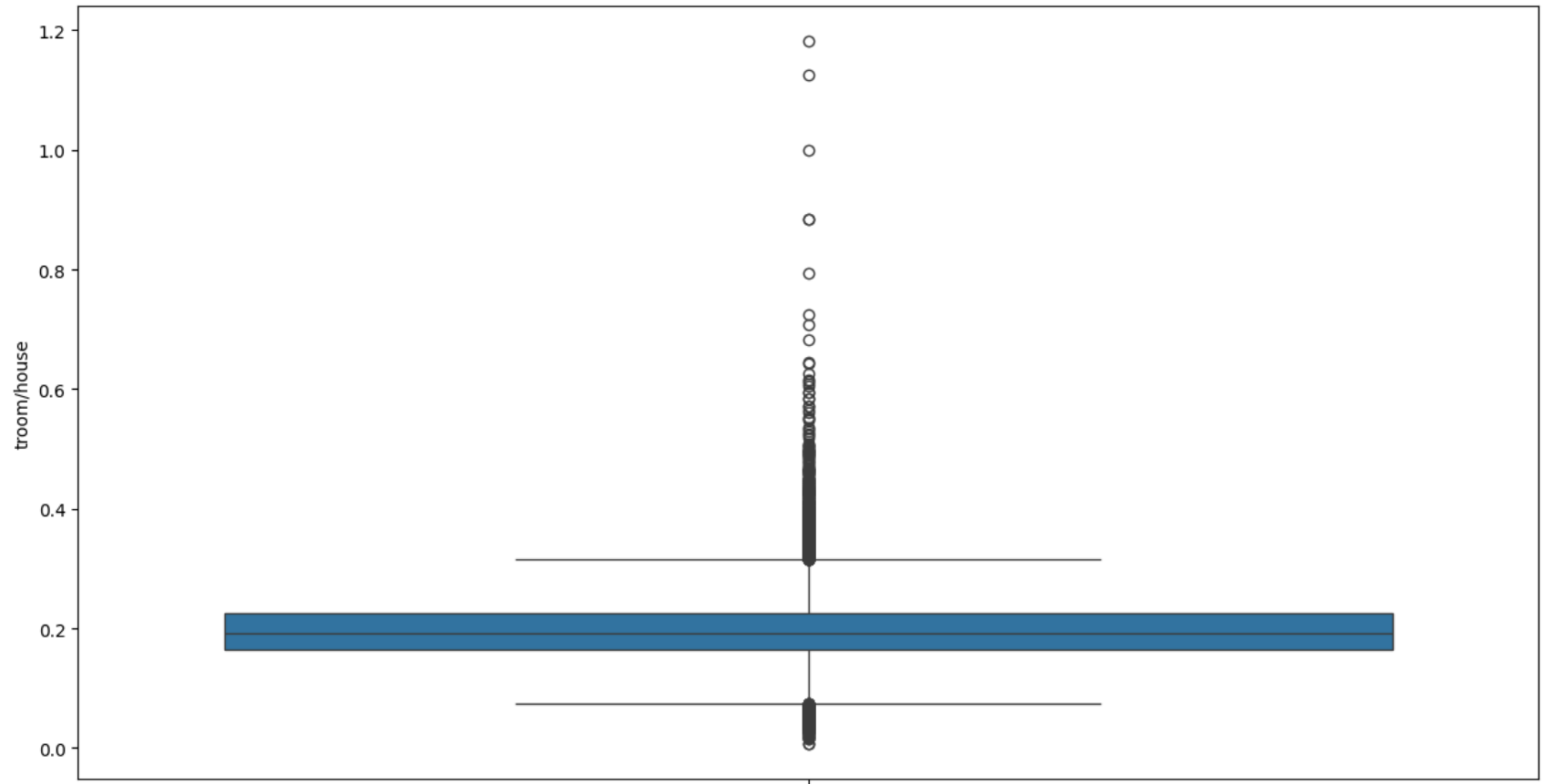


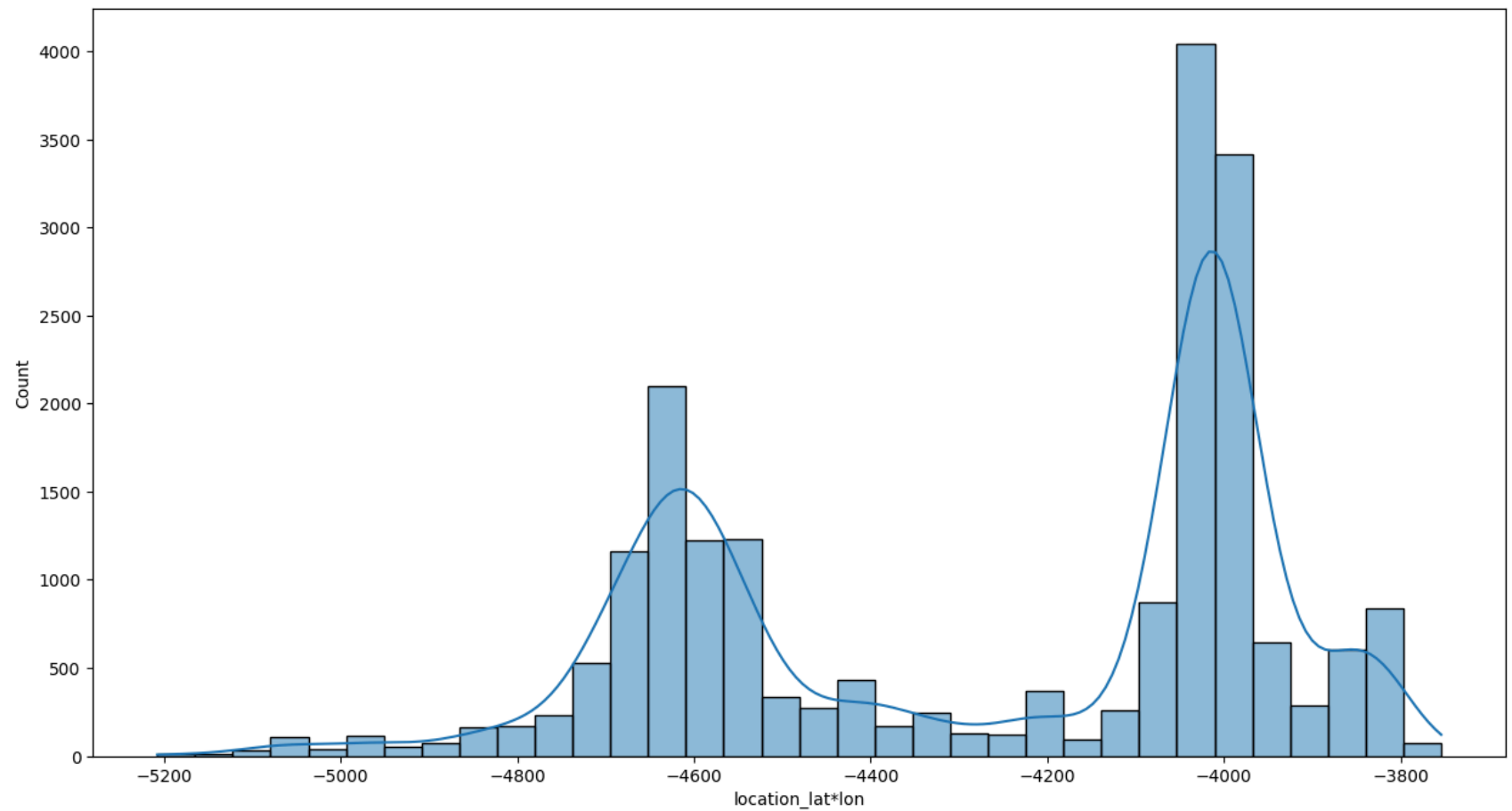


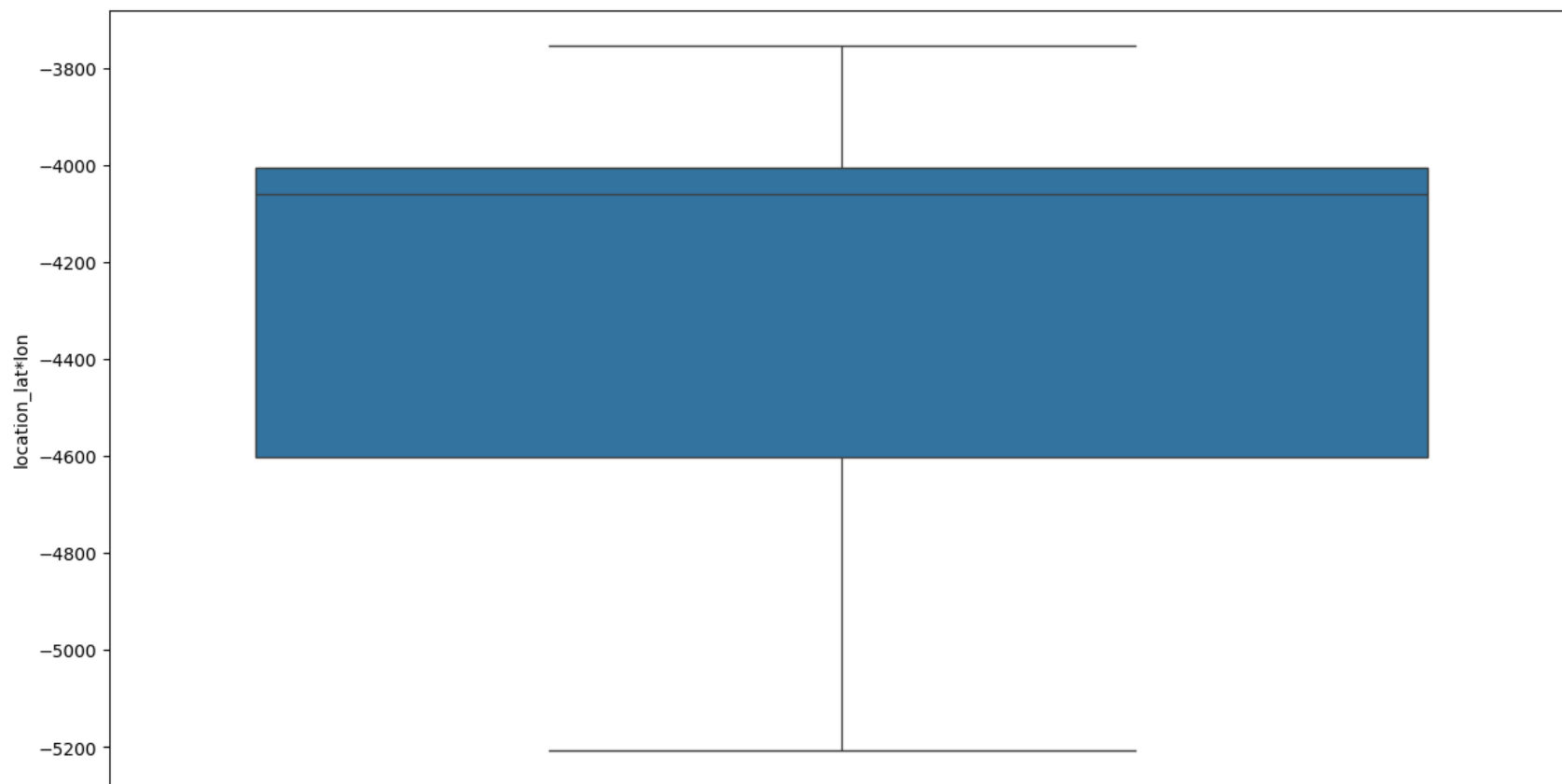












```
In [ ]: df['total_bedrooms'] = np.log1p(df['total_bedrooms'] +1)
df['total_rooms'] = np.log1p(df['total_rooms']+1)
df['population'] = np.log1p(df['population']+1)
df['households'] = np.log1p(df['households']+1)
df['median_income'] = np.log1p(df['median_income']+1)
```

```
In [ ]: X =df.drop('median_house_value', axis=1)
y= df[['median_house_value']]
```

```
In [ ]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error , r2_score
```

```
In [ ]: X_train , X_test , y_train , y_test = train_test_split(X , y , test_size=0.2 , random_state=42 )
```

```
In [ ]: from sklearn.model_selection import RandomizedSearchCV

rf= RandomForestRegressor()
```

```
In [ ]: param_dist ={
    'n_estimators' : [100,200,300,450],
    'max_depth' : [10,20,30,35],
    'min_samples_split':[ 2,4,8,9],
    'min_samples_leaf' : [1,2,3,4]
}

random_search = RandomizedSearchCV(
    rf,
    param_distributions=param_dist,
    n_iter=10,
    cv=3,
    scoring='neg_mean_absolute_error',
    n_jobs=-1,
    random_state=42
)

random_search.fit(X_train, y_train)

print("Best Random Params:", random_search.best_params_)
```

c:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\base.py:1336: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return fit_method(estimator, *args, **kwargs)
```

Best Random Params: {'n_estimators': 450, 'min_samples_split': 4, 'min_samples_leaf': 3, 'max_depth': 30}

```
In [ ]: from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [400,425,450],
    'max_depth': [20,30,40],
    'min_samples_split': [3,4,5],
    'min_samples_leaf': [2,3,4]
}
```

```

grid_search = GridSearchCV(
    rf,
    param_grid=param_grid,
    cv=3,
    scoring='neg_mean_absolute_error',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print("Best Grid Params:", grid_search.best_params_)

```

c:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\base.py:1336: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return fit_method(estimator, *args, **kwargs)
```

Best Grid Params: {'max_depth': 30, 'min_samples_leaf': 2, 'min_samples_split': 4, 'n_estimators': 425}

```

In [ ]: Scaler = RobustScaler()

X_train = Scaler.fit_transform(X_train)
X_test = Scaler.transform(X_test)

y_train = np.log1p(y_train)
y_test = np.log1p(y_test)

```

```

In [ ]: model_linear = LinearRegression()
model_random = RandomForestRegressor( n_estimators=425,
    max_depth=30,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42)

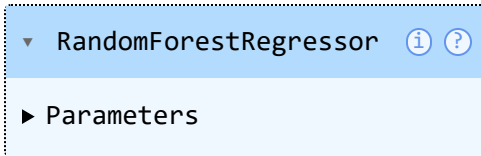
model_linear.fit(X_train , y_train)
model_random.fit(X_train , y_train)

```

c:\Users\Admin\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\base.py:1336: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return fit_method(estimator, *args, **kwargs)
```

Out[]:



```
In [ ]: y_pred_linear = model_linear.predict(X_test)
        y_pred_random = model_random.predict(X_test)
```

```
In [ ]: from sklearn.metrics import r2_score

        y_pred = model_random.predict(X_test)

        # inverse log
        y_test_inv = np.exp1(y_test)
        y_pred_inv = np.exp1(y_pred)

        print("Real R2:", r2_score(y_test_inv, y_pred_inv))
```

Real R2: 0.8212968981574231

```
In [ ]: from sklearn.metrics import mean_absolute_error

        MAE_random = mean_absolute_error(y_test_inv , y_pred_inv)
```

```
In [ ]: print(MAE_random)
```

30937.90586938801

```
In [ ]: print(model_linear.score(X_test , y_test))
        print(model_random.score(X_test , y_test))
```

0.7165064326488995
0.8453116011722315

```
In [ ]:
```